Relatório RSA

Nome: José Henrique Ioki Yamaoki

História do RSA

A criptografia RSA é um dos métodos de criptografia de chave pública mais conhecidos e utilizados na história da computação. Seu nome é formado pelas iniciais de seus inventores, Ronald Rivest, Adi Shamir e Leonard Adleman, que desenvolveram o algoritmo em 1977 e publicaram seu trabalho em 1978, no Massachusetts Institute of Technology (MIT). O RSA utiliza um par de chaves, uma pública e outra privada, permitindo que dados sejam criptografados com uma chave e descriptografados com a outra.

Durante a popularização da Internet, o RSA também foi utilizado em protocolos de segurança para proteger comunicações. O Netscape Navigator, um dos navegadores mais populares da década de 1990, utilizava criptografia RSA como parte do sistema de segurança empregado nas comunicações por HTTPS. Dessa forma, o RSA teve um papel importante no desenvolvimento da segurança na Internet, especialmente na proteção da troca de informações entre navegadores e servidores.

Importância na criptografia moderna

O RSA está presente em diversos softwares usados atualmente.
Exemplos:

- Certificados e assinaturas digitais: Desde muito tempo que as autoridades certificadoras utilizam o RSA para assinar documentos, com essas assinaturas é possível verificar a autenticidade de documentos e identidades.
- TLS/SSL: O RSA é usado no processo de handshake TLS que protege sites HTTPS, tanto para troca de chaves quanto para autenticação.
- VPNs: Dependem do RSA para trocar chaves e verificar a identidade.

Explicação matemática

O algoritmo por trás do RSA se baseia em uma assimetria simples, é muito rápido fazer a multiplicação entre dois números primos, porém fatorar esse produto com valores de números primos muito grandes ainda é demorado para o poder de processamento que temos atualmente.

Em alto nível, o RSA precisa de uma chave primária e uma chave pública usando dois números primos, com essas chaves um ponto A pode cifrar o conteúdo com a chave pública de B, e B pode decifrar o conteúdo usando sua própria chave privada e vice-versa.

Geração de números primos

Uma das maneiras mais simples de gerar um número primo, é escolher de forma aleatória um número bem alto e a partir dele ir testando a primalidade de cada número após ele até encontrar um número que seja primo.

Para verificar se um número é primo, um algoritmo comum para teste de primalidade é o seguinte:

Algoritmo de fatoração
Entrada: inteiro positivo n.
Saída: verdadeiro se n for primo, caso contrário, falso.
Etapa 1: Comece fazendo f = 2.
Etapa 2: Se n/f é inteiro, retorna falso, senão vá para a Etapa 3.
Etapa 3: Incremente f de uma unidade e vá para a Etapa 4.
Etapa 4: Se f > $\sqrt{n}$ retorna verdadeiro; senão volte à Etapa 2.

Cálculo do módulo n

O módulo n é o produto de dois primos grandes e é a base matemática que determina o espaço de cálculo do RSA. A segurança vem, em grande parte, da dificuldade de descobrir p e q conhecendo apenas n.
Em aritmética modular, dois números são congruentes módulo n, se a - b é um múltiplo de n. Em outras palavras, se o resto da divisão de a por n for igual ao resto da divisão de b por n. Exemplo:

n = 5
$10 \equiv 20\ (mod\ 5)$ e $13 \equiv 23\ (mod\ 5)$

Função Totiente de Euler

A Função Totiente de Euler, representada por $\varphi(n)$, é uma função matemática muito importante no funcionamento do algoritmo RSA. Ela determina a quantidade de números inteiros positivos menores que n que são coprimos com n, ou seja, que não possuem nenhum divisor comum com n, além de 1.
Como $n = pq$,

$$\varphi(n) = (p - 1)(q - 1)$$

No RSA, esse valor é fundamental porque é utilizado para escolher o expoente público e e calcular o expoente privado d.

Escolha do expoente público (e)

O valor de e é escolhido depois do cálculo da função totiente de Euler $\varphi(n)$. Para que ele seja válido, deve satisfazer duas condições principais:
$1 < e < \varphi(n)$ e $gcd(e, \varphi(n)) = 1$
O expoente e é utilizado durante a criptografia da mensagem. Posteriormente, ele também será utilizado para calcular o expoente privado d, que é necessário para a descriptografia.

Cálculo do expoente privado (d)

O expoente privado d é uma das partes mais importantes da chave privada do RSA. Ele é utilizado no processo de descriptografia, permitindo que a mensagem criptografada seja recuperada.

O valor de d é calculado a partir do expoente público e e da função totiente de Euler $\varphi(n)$. Matematicamente, d deve satisfazer:

$$d \times e \equiv 1 (mod\, \varphi(n))$$

Durante a descriptografia, utiliza-se d para recuperar a mensagem original:

$$m = c^d \mod n$$

Criptografia

Antes de começar o processo de criptografia em si, é necessário realizar a etapa de pré-codificação, que consiste em agrupar os dados da mensagem em blocos.

Exemplo:
Mensagem: “HELLO”
UTF-8: 48 45 4C 4C 4F

Separando em blocos de 16 bits: 4845 4c4c 4f00

Em seguida, são gerados dois números primos, p e q.

Com base nesses valores, são determinados os expoentes público e privado, conforme explicado anteriormente. O par (n, e) constitui a chave pública.

Para codificar um bloco, basta usar a seguinte fórmula: $C(b) = b^e \mod n$
O resultado $C(b)$ é o bloco cifrado correspondente. Esse processo é repetido para cada bloco da mensagem.

Descriptografia

Para realizar a descriptografia, são utilizados os mesmos valores n e o expoente privado d, determinados durante a geração das chaves. A chave privada é formada pelo par (n, d).

Para descriptografar um bloco cifrado a, basta utilizar a seguinte fórmula:

$$D(a) = a^d \, mod \, n$$

O resultado $D(a)$ corresponde ao bloco original antes da criptografia. Esse processo é repetido para cada bloco cifrado da mensagem.

Após a descriptografia de todos os blocos, os valores obtidos são convertidos novamente para bytes e, em seguida, para caracteres utilizando UTF-8. Dessa forma, os blocos originais são reconstruídos e a mensagem "HELLO" é recuperada.

Análise de segurança

O RSA é considerado um algoritmo de criptografia assimétrica seguro quando implementado corretamente e utilizando parâmetros adequados. Sua segurança está baseada principalmente na dificuldade computacional de fatorar um número inteiro grande em seus dois fatores primos.

A chave pública contém n e e, enquanto a chave privada utiliza d. Embora n seja divulgado publicamente, obter p e q a partir de n deve ser computacionalmente inviável quando os primos possuem tamanho adequado. Conhecendo p e q, entretanto, seria possível calcular o totiente de n e, consequentemente, determinar o expoente privado d. Portanto, a proteção dos fatores primos é fundamental para a segurança da chave privada.

Outro aspecto importante é o tamanho da chave. Chaves muito pequenas podem ser quebradas por técnicas de fatoração com recursos computacionais modernos. Por isso, implementações atuais normalmente utilizam chaves RSA de 2048 bits ou mais, dependendo do nível de segurança desejado.

Entretanto, a segurança do RSA não depende apenas do tamanho da chave. A forma como o algoritmo é utilizado também é fundamental. O RSA "puro", aplicando diretamente C = b^e mod n sobre os dados, possui propriedades indesejáveis e pode ser vulnerável a determinados ataques. Em aplicações reais, é necessário utilizar esquemas de padding apropriados, como o RSA-OAEP, para criptografia.

Além disso, RSA é vulnerável se a implementação apresentar problemas como geração previsível de números aleatórios, reutilização inadequada de chaves, exposição da chave privada ou implementação suscetível a ataques de canal lateral, como ataques baseados no tempo de execução ou no consumo de energia.

Por fim, o RSA não é considerado resistente a computadores quânticos de grande escala. Um computador quântico suficientemente poderoso poderia utilizar o algoritmo de Shor para fatorar n de maneira muito mais eficiente, comprometendo a segurança do RSA. Por esse motivo, existem atualmente pesquisas e padrões voltados à adoção de algoritmos de criptografia pós-quântica.